

条款 36：绝不重新定义继承而来的 non-virtual 函数

Never redefine an inherited non-virtual function.

假设我告诉你，class D 系由 class B 以 public 形式派生而来，class B 定义有一个 public 成员函数 mf。由于 mf 的参数和返回值都不重要，所以我假设两者皆为 void。换句话说我的意思是：

```
class B {
public:
    void mf();
    ...
};

class D: public B { ... };
```

虽然我们对 B、D 和 mf 一无所知，但面对一个类型为 D 的对象 x：

```
D x;           //x 是一个类型为 D 的对象
```

如果以下行为：

```
B* pB = &x;           //获得一个指针指向 x
pB->mf();              //经由该指针调用 mf
```

异于以下行为：

```
D* pD = &x;           //获得一个指针指向 x
pD->mf();              //经由该指针调用 mf
```

你可能会相当惊讶。毕竟两者都通过对象 x 调用成员函数 mf。由于两者所调用的函数都相同，凭借的对象也相同，所以行为也应该相同，是吗？

是的，理应如此，但事实可能不是如此。更明确地说，如果 mf 是个 non-virtual 函数而 D 定义有自己的 mf 版本，那就不是如此：

```
class D: public B {
public:
    void mf();           //遮掩 (hides) 了 B::mf; 见条款 33
    ...
};

pB->mf();                //调用 B::mf
pD->mf();                //调用 D::mf
```

造成此一两面行为的原因是，non-virtual 函数如 B::mf 和 D::mf 都是静态绑定 (statically bound, 见条款 37)。这意思是，由于 pB 被声明为一个 pointer-to-B，通

过 `pB` 调用的 non-virtual 函数永远是 `B` 所定义的版本, 即使 `pB` 指向一个类型为 “`B` 派生之 class” 的对象, 一如本例。

但另一方面, virtual 函数却是动态绑定 (dynamically bound, 见条款 37), 所以它们不受这个问题之苦。如果 `mf` 是个 virtual 函数, 不论是通过 `pB` 或 `pD` 调用 `mf`, 都会导致调用 `D::mf`, 因为 `pB` 和 `pD` 真正指的都是一个类型为 `D` 的对象。

如果你正在编写 class `D` 并重新定义继承自 class `B` 的 non-virtual 函数 `mf`, `D` 对象很可能展现出精神分裂的不一致行径。更明确地说, 当 `mf` 被调用, 任何一个 `D` 对象都可能表现出 `B` 或 `D` 的行为; 决定因素不在对象自身, 而在于 “指向该对象之指针” 当初的声明类型。References 也会展现和指针一样难以理解的行径。

但那只是实务面上的讨论。我知道你真正想要的是理论层面的理由 (关于 “绝不重新定义继承而来的 non-virtual 函数” 这回事)。我很乐意为你服务。

条款 32 已经说过, 所谓 public 继承意味 **is-a** (是一种) 的关系。条款 34 则描述为什么在 class 内声明一个 non-virtual 函数会为该 class 建立起一个不变性 (invariant), 凌驾其特异性 (specialization)。如果你将这两个观点施行于两个 classes `B` 和 `D` 以及 non-virtual 成员函数 `B::mf` 身上, 那么:

- 适用于 `B` 对象的每一件事, 也适用于 `D` 对象, 因为每个 `D` 对象都是一个 `B` 对象;
- `B` 的 derived classes 一定会继承 `mf` 的接口和实现, 因为 `mf` 是 `B` 的一个 non-virtual 函数。

现在, 如果 `D` 重新定义 `mf`, 你的设计便出现矛盾。如果 `D` 真有必要实现出与 `B` 不同的 `mf`, 并且如果每一个 `B` 对象——不管多么特化——真的必须使用 `B` 所提供的 `mf` 实现码, 那么 “每个 `D` 都是一个 `B`” 就不为真。既然如此 `D` 就不该以 public 形式继承 `B`。另一方面, 如果 `D` 真的必须以 public 方式继承 `B`, 并且如果 `D` 真有需要实现出与 `B` 不同的 `mf`, 那么 `mf` 就无法为 `B` 反映出 “不变性凌驾特异性” 的性质。既然如此 `mf` 应该声明为 virtual 函数。最后, 如果每个 `D` 真的是一个 `B`, 并且如果 `mf` 真的为 `B` 反映出 “不变性凌驾特异性” 的性质, 那么 `D` 便不需要重新定义 `mf`, 而且它也不应该尝试这样做。

不论哪一个观点, 结论都相同: 任何情况下都不该重新定义一个继承而来的 non-virtual 函数。

如果此条款使你感到枯燥乏味，或许是因为你已经读过条款 7，该条款解释为什么多态性（polymorphic）base classes 内的析构函数应该是 virtual。如果你违反那个准则（也就是说如果你在 polymorphic base class 内声明一个 non-virtual 析构函数），你也就违反了本条款，因为 derived classes 绝对不该重新定义一个继承而来的 non-virtual 函数（此处指的是 base class 析构函数）。即使没有声明析构函数，此亦为真，因为条款 5 说，析构函数是“如果你没有为自己声明一个，编译器会为你生成一个”的数种成员函数之一。就本质而言，条款 7 只不过是本条款的一个特殊案例，尽管它也足够重要到单独成为一个条款。

请记住

■ 绝对不要重新定义继承而来的 non-virtual 函数。

条款 37：绝不重新定义继承而来的缺省参数值

Never redefine a function's inherited default parameter value.

让我们一开始就将讨论简化。你只能继承两种函数：virtual 和 non-virtual 函数。然而重新定义一个继承而来的 non-virtual 函数永远是错误的（见条款 36），所以我们可以安全地将本条款的讨论局限于“继承一个带有缺省参数值的 virtual 函数”。

这种情况下，本条款成立的理由就非常直接而明确了：virtual 函数系动态绑定（dynamically bound），而缺省参数值却是静态绑定（statically bound）。

那是什么意思？你说你那负荷过重的脑袋早已忘记静态绑定和动态绑定之间的差异？（为了正式记录在案，容我再说一次，静态绑定又名前期绑定，*early binding*；动态绑定又名后期绑定，*late binding*。）现在让我们来一趟复习之旅吧！

对象的所谓静态类型（static type），就是它在程序中被声明时所采用的类型。考虑以下的 class 继承体系：

```
//一个用以描述几何形状的 class
class Shape {
public:
    enum ShapeColor { Red, Green, Blue };
    //所有形状都必须提供一个函数，用来绘出自己
    virtual void draw(ShapeColor color = Red) const = 0;
    ...
};
```

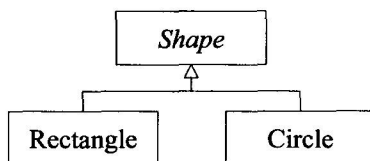
```

class Rectangle: public Shape {
public:
    //注意, 赋予不同的缺省参数值。这真糟糕!
    virtual void draw(ShapeColor color = Green) const;
    ...
};

class Circle: public Shape {
public:
    virtual void draw(ShapeColor color) const;
    //译注: 请注意, 以上这么写则当客户以对象调用此函数, 一定要指定参数值。
    //      因为静态绑定下这个函数并不从其 base 继承缺省参数值。
    //      但若以指针 (或 reference) 调用此函数, 可以不指定参数值,
    //      因为动态绑定下这个函数会从其 base 继承缺省参数值。
    ...
};

```

这个继承体系图示如下:



现在考虑这些指针:

```

Shape* ps;           //静态类型为 Shape*
Shape* pc = new Circle; //静态类型为 Shape*
Shape* pr = new Rectangle; //静态类型为 Shape*

```

本例中 `ps`, `pc` 和 `pr` 都被声明为 `pointer-to-Shape` 类型, 所以它们都以它为静态类型。注意, 不论它们真正指向什么, 它们的静态类型都是 `Shape*`。

对象的所谓动态类型 (dynamic type) 则是指“目前所指对象的类型”。也就是说, 动态类型可以表现出一个对象将会有何行为。以上例而言, `pc` 的动态类型是 `Circle*`, `pr` 的动态类型是 `Rectangle*`。`ps` 没有动态类型, 因为它尚未指向任何对象。

动态类型一如其名称所示, 可在程序执行过程中改变 (通常是经由赋值动作):

```

ps = pc;           //ps 的动态类型如今是 Circle*
ps = pr;           //ps 的动态类型如今是 Rectangle*

```

`Virtual` 函数系动态绑定而来, 意思是调用一个 `virtual` 函数时, 究竟调用哪一份函数实现代码, 取决于发出调用的那个对象的动态类型:

```
pc->draw(Shape::Red);           //调用 Circle::draw(Shape::Red)
pr->draw(Shape::Red);           //调用 Rectangle::draw(Shape::Red)
```

我知道这些都是老调重弹；是的，你当然已经了解 `virtual` 函数。但是当你考虑带有缺省参数值的 `virtual` 函数，花样来了，因为就如我稍早所说，`virtual` 函数是动态绑定，而缺省参数值却是静态绑定。意思是你可能会在“调用一个定义于 `derived` class 内的 `virtual` 函数”的同时，却使用 `base` class 为它所指定的缺省参数值：

```
pr->draw( );                     //调用 Rectangle::draw(Shape::Red) !
```

此例之中，`pr` 的动态类型是 `Rectangle*`，所以调用的是 `Rectangle` 的 `virtual` 函数，一如你所预期。`Rectangle::draw` 函数的缺省参数值应该是 `GREEN`，但由于 `pr` 的静态类型是 `Shape*`，所以此一调用的缺省参数值来自 `Shape` class 而非 `Rectangle` class！结局是这个函数调用有着奇怪并且几乎绝对没人预料得到的组合，由 `Shape` class 和 `Rectangle` class 的 `draw` 声明式各出一半力。

以上事实不只局限于“`ps`, `pc` 和 `pr` 都是指针”的情况；即使把指针换成 `references` 问题仍然存在。重点在于 `draw` 是个 `virtual` 函数，而它有个缺省参数值在 `derived` class 中被重新定义了。

为什么 C++ 坚持以这种乖张的方式来运作呢？答案在于运行期效率。如果缺省参数值是动态绑定，编译器就必须有某种办法在运行期为 `virtual` 函数决定适当的参数缺省值。这比目前实行的“在编译期决定”的机制更慢而且更复杂。为了程序的执行速度和编译器实现上的简易度，C++ 做了这样的取舍，其结果就是你如今所享受的执行效率。但如果你没有注意本条款所揭示的忠告，很容易发生混淆。

这一切都很好，但如果你试着遵守这条规则，并且同时提供缺省参数值给 `base` 和 `derived` classes 的用户，又会发生什么事呢？

```
class Shape {
public:
    enum ShapeColor { Red, Green, Blue };
    virtual void draw(ShapeColor color = Red) const = 0;
    ...
};
```

```
class Rectangle: public Shape {
public:
    virtual void draw(ShapeColor color = Red) const;
    ...
};
```

喔欧，代码重复。更糟的是，代码重复又带着相依性（with dependencies）：如果 Shape 内的缺省参数值改变了，所有“重复给定缺省参数值”的那些 derived classes 也必须改变，否则它们最终会导致“重复定义一个继承而来的缺省参数值”。怎么办？

当你想令 virtual 函数表现出你所想要的行为但却遭遇麻烦，聪明的做法是考虑替代设计。条款 35 列了不少 virtual 函数的替代设计，其中之一是 NVI (*non-virtual interface*) 手法：令 base class 内的一个 public non-virtual 函数调用 private virtual 函数，后者可被 derived classes 重新定义。这里我们可以让 non-virtual 函数指定缺省参数，而 private virtual 函数负责真正的工作：

```
class Shape {
public:
    enum ShapeColor { Red, Green, Blue };
    void draw(ShapeColor color = Red) const           //如今它是 non-virtual
    {
        doDraw(color);                               //调用一个 virtual
    }
    ...
private:
    virtual void doDraw(ShapeColor color) const = 0; //真正的工作
};                                                    //在此处完成

class Rectangle: public Shape {
public:
    ...
private:
    virtual void doDraw(ShapeColor color) const;      //注意，不须指定
    ...                                              //缺省参数值。
};
```

由于 non-virtual 函数应该绝对不被 derived classes 覆写（见条款 36），这个设计很清楚地使得 draw 函数的 color 缺省参数值总是为 Red。

请记住

- 绝对不要重新定义一个继承而来的缺省参数值，因为缺省参数值都是静态绑定，而 virtual 函数——你唯一应该覆写的东西——却是动态绑定。